

Module 10 — Custom Geometry & Sensitive Detectors

Supplementary to the sPHENIX Standalone Course

1. Four pillars of a Geant4 simulation

Every Geant4 application, from a medical dosimeter to sPHENIX, is built from four user-extended classes:

	Base class	Responsibility
	G4VUserDetectorConstruction	What materials exist, where
	G4VModularPhysicsList	Which interactions are simulated
	G4VUserPrimaryGeneratorAction	Primary particle(s) per event
	G4UserSteppingAction / G4VSensitiveDetector	What to record

2. Materials

Use G4NistManager for pre-defined materials. Build custom ones from elements or existing materials:

```
auto nist = G4NistManager::Instance();
G4Material* air = nist->FindOrBuildMaterial("G4_AIR");
G4Material* pb = nist->FindOrBuildMaterial("G4_Pb");
G4Material* lAr = nist->FindOrBuildMaterial("G4_lAr");

// Polystyrene proxy: C8H8,  $\rho=1.032 \text{ g/cm}^3$ 
auto H = nist->FindOrBuildElement("H");
auto C = nist->FindOrBuildElement("C");
auto scint = new G4Material("Scint", 1.032*g/cm3, 2);
scint->AddElement(C, 8);
scint->AddElement(H, 8);
```

3. Volumes: solid, logical, physical

Geant4 separates shape, properties, and placement:

Class	What it carries
G4VSolid	Shape only (box, tube, sphere, ...)
G4LogicalVolume	Solid + material + optionally sensitive detector, region, field
G4VPhysicalVolume	Placement of a logical volume in a mother

Note: Mother-daughter rule: a daughter volume must fit entirely within its mother. Overlaps are silent — always run `/geometry/test/run` in a macro during development.

4. Replicas for periodic structures

Avoid placing each layer individually:

```
auto layerS = new G4Tubs("Layer", 80*cm, 120*cm, 2*cm, 0, 360*deg);
auto layerL = new G4LogicalVolume(layerS, lAr, "Layer");
new G4PVReplica("Layer", layerL, calL, kZAxis, 50, 4*cm);
```

Each replica has a unique copyNumber accessible in scoring code via GetTouchableHandle()->GetCopyNumber().

5. Magnetic fields

The simplest field is uniform and attached to a tracking volume:

```
G4ThreeVector fieldVec(0, 0, 1.4*tesla);
auto magField = new G4UniformMagField(fieldVec);
auto fieldMgr = new G4FieldManager(magField);
fieldMgr->CreateChordFinder(magField);
trackerLogical->SetFieldManager(fieldMgr, true);
```

Non-uniform fields inherit from G4MagneticField and override GetFieldValue:

```
class SolenoidMap : public G4MagneticField {
public:
    void GetFieldValue(const G4double point[4],
                      G4double* B) const override {
        double r = std::hypot(point[0], point[1]);
        double z = point[2];
        // interpolate Br(r,z), Bz(r,z) from a tabulated map ...
        B[0] = bx; B[1] = by; B[2] = bz;
    }
};
```

6. Regions and production cuts

Region-specific cuts let you spend CPU where it counts:

```
auto calRegion = new G4Region("CalRegion");
```

```
calL->SetRegion(calRegion);  
calRegion->AddRootLogicalVolume(calL);  
  
auto cuts = new G4ProductionCuts();  
cuts->SetProductionCut(0.1*mm, "gamma");  
cuts->SetProductionCut(0.1*mm, "e-");  
cuts->SetProductionCut(0.1*mm, "e+");  
calRegion->SetProductionCuts(cuts);
```

7. Sensitive detectors

Instead of manual bookkeeping in a stepping action, a `G4VSensitiveDetector` produces `G4VHits` that are collected per event:

```
class CalHit : public G4VHit {
public:
    G4double edep = 0;
    G4int     layerID = -1;
};

class CalSD : public G4VSensitiveDetector {
public:
    CalSD(const G4String& n) : G4VSensitiveDetector(n) {
        collectionName.insert("CalHits");
    }
    void Initialize(G4HCofThisEvent* hcof) override {
        fCol = new G4THitsCollection<CalHit>(SensitiveDetectorName,
                                             collectionName[0]);

        hcof->AddHitsCollection(
            G4SDManager::GetSDMpointer()->GetCollectionID(collectionName[0]),
            fCol);
    }
    G4bool ProcessHits(G4Step* step, G4TouchableHistory*) override {
        double edep = step->GetTotalEnergyDeposit();
        if (edep <= 0) return false;
        auto copy = step->GetPreStepPoint()
                    ->GetTouchableHandle()->GetCopyNumber();
        auto* h = new CalHit; h->edep = edep; h->layerID = copy;
        fCol->insert(h);
        return true;
    }
private:
    G4THitsCollection<CalHit>* fCol = nullptr;
};

// In DetectorConstruction::ConstructSDandField():
auto sd = new CalSD("CalSD");
G4SDManager::GetSDMpointer()->AddNewDetector(sd);
SetSensitiveDetector("Layer", sd);
```

8. Debug recipes

Problem	First thing to try
SD reports zero hits	Check logical-volume name matches <code>SetSensitiveDetector</code>
Tracks bend wrongly	Print <code>fieldMgr</code> to confirm it's attached
Overlap warning	Run <code>/geometry/test/run</code> and fix mother-daughter gap
Slow runs	Loosen production cuts in non-sensitive regions

9. Sanity tests to run first

Before any physics run, validate geometry:

```

/run/initialize
/geometry/test/run      # overlap check
/vis/drawVolume
/vis/viewer/flush
/gun/particle e-
/gun/energy 1 GeV
/run/beamOn 1
# verify a single event's tracks look reasonable in the viewer

```

10. Where to go next

Once you can build custom geometry: write a sensitive detector, add a field, and run a longitudinal or transverse shower study. Those three exercises cover 90% of what's needed for a detector R&D; study at the simulation level.